Meeting on 25 /7

Wolfgang, Gawtam, Sara, Thomas
—

Main Points

a/ Final Hackathon Breakdown

b/ Invitation to Find Hackathons

c/ Everyone's Timeline for a Startup

Meeting on 18 /7

Wolfgang, Gawtam, Thomas
—

Main Points

a/ Understanding the Porsche Guy's Project

b/ See if ABB likes the Demo.
Brainstorm

- Making a Hardware project simpler by using AI to decomplexify
- AI is probably just a demo towards a PLM project
- A production side debugger
- Helping with Maintenance / Monitoring the System for Failures / Edge Cases.

Edit the Demo if needed - Gawtam lets Thomas know

c/ Kickmaker building Physical AI on Factory Floors

d/ Could speak to (Robotics, Medical) Startups in Scaling Phase (scale-up)

- Central Nervous System (cardiovascular, lymphatic, cerebrospinal fluids//). Monitor the movement of fluid to recognize edge cases (pathologies)

Pitch (gen. by Claude)

Connect a company's product requirements to their live robots, so that when a test fails, the system returns a cited root-cause diagnosis instead of a raw error.

Problem

Physical AI is moving from labs into real deployments, and two things are true at once:

Requirements are exhaustively documented. PLM systems, safety cases, and requirement trackers define what a machine must do — often under regulatory obligation.

Testing against those requirements is manual and disconnected. When a robot misbehaves, telemetry produces a *symptom* — "joint overshoots," "gripper slips under load," "gait destabilizes on incline." A human engineer then reads logs, recalls institutional knowledge, and manually maps that symptom to the requirement it violates and the known issue behind it.

That mapping step is where the time goes, where tribal knowledge lives, and where things break when the engineer who knew it leaves. It doesn't scale either: test suites and fleets generate far more symptoms than engineers can triage, and every hardware revision resets the knowledge.

Solution

A symptom is not a root cause. The gap between them is a *knowledge* problem, not a sensing problem — and requirement documents already contain that knowledge.

An agent holds the requirements as durable memory and can query the live machine directly through its middleware. When a test fails, it maps the live symptom to the requirement it violates, the root cause, the fix, and the known-issue tag — and cites every one of them.

Citation is the product, not a feature. In safety-critical hardware, an uncited diagnosis is unusable: engineers won't act on it and auditors won't accept it.

Proof of concept

Validated end-to-end.

> The PoC runs on a self-driving car's steering subsystem. That was scaffolding — a subsystem we could simulate, fault-inject, and validate in 12hr**.**

Live system integration. A real ROS 2 control stack with a fault injector supporting five distinct fault modes (bias, reversed, stuck, noise, lag).

- **Symptom detection.** A pure, unit-tested classifier compares commanded intent against measured response over a rolling window and emits a symptom with a health status — never a root cause. Twelve tests, one per decision branch.
- **Agent access to the machine.** An MCP server exposes the live system to an LLM agent as callable tools: read health, read stats, inject fault, restart unit.
- **Requirement memory.** Requirements and known issues held as durable agent memory, mapped from symptom to requirement ID, root cause, fix, and known-issue tag.
- **Closed loop.** A web panel shows live telemetry, a cited diagnosis, and a one-click fix that round-trips through the real stack until the test flips to PASS.

One design choice worth preserving: the fault injector is named like a production ECU, so the agent can't shortcut the diagnosis by reading node names. It has to reason from the signal. That constraint is what separates this from a demo trick.

Open questions

- **Requirement ingestion.** Most requirements live in unstructured Word, Jira, Confluence, and PDF — not clean structured stores. Is ingestion a solvable onboarding step, or is it the actual product?
- **Where the value lands.** Pre-deployment testing and CI, or deployed-fleet triage? Different products.
- **Classifier scaling.** Reliable, low-false-positive symptom classifiers are the hardest asset to build and the most defensible. But do they need rebuilding per customer, per subsystem? If so, is this a product or a consultancy?
- **Deployment model.** On-prem or self-hosted is probably mandatory given IP sensitivity. What does that cost in iteration speed?
- **The honest one.** Would a team just prompt an LLM with their requirement doc and their logs and get 70% of this for free? What makes the remaining 30% worth paying for? *(Current answer: determinism, live system access, and citations an auditor will accept. Needs testing against users, not asserting.)*

Kelvin Yee :

Occupation : Test Engineer
Seniority : Junior

Keywords : HITL (Hardware in the Loop)

Note from Thomas — Understanding Kelvin's Use of Cursor

Even with under two years of seniority, Kelvin should be able to articulate what he actually uses Cursor for day to day — and it may not match our assumptions.

Open question on system testing scope:
While we're building modularity for full-system testing, I'm skeptical that all systems are tested **simultaneously** across PLM and live deployment. Even if they are, can we reliably isolate one system's fault from another's, or does fault correlation make that hard to guarantee? This likely varies a lot by application, so we should ask him directly rather than assume this about modularity.

Questions for Kelvin (or another test engineer):

Daily workflow

- What does he actually do on a daily basis?
- What friction points does he run into — logistically, in team dynamics, or in time-to-complete-task?

Tooling potential

- I've heard test engineers also design their own control systems. If true, this could position Cursor as a design tool for them, not just a testing aid — worth exploring as a future use case.

Access and permissions

- Does he have access to modify low-level sensors?
- Does he have access to modify high-level PLM systems?

A discussion with my biomedical director brought up this same bottleneck.

Organizational context

- Which department at Porsche does he sit in, and what does that department own?
- What's his specific role/responsibility within his team?

Data

- Can he (or his team) share sample/dummy data in their actual format? This would give us a much more concrete picture of what their work looks like in practice.

Next steps

- I can reach out to another test engineer for a second perspective.
- Some test engineers post on YouTube — worth checking if any public channels give us insight before we even need to ask.

Lead currently pursued by: Thomas

Unknown : through ST-Bio Lab Director

Occupation : PLM Expert
Seniority : ?

Keywords : PLM, Sensors, Biotech

NVIDIA HITL
Hardware-in-the-Loop (HIL) Fundamentals — Getting Started With Isaac Sim

CloudBees : SITL ? Definitely similar to our product in many ways
Link

Foxglove : https://foxglove.dev/

- Important and established company for Physical AI data visualization.
- Would be a good compliment / “integration” to our tool.

Thomas :

-

Wolf :

Gawtam :

- NVIDIA research announced /agents for physical AI.
    - Agents understand and make physical AI efficient
    - Our product (cursor “agent”) could be the “doctor” of these agents.

- What would be the MVP
- How would it compare with NVIDIA’s Digital Twins?
- Agentic workflows for Robotics
-

Thomas

Wolfgang

Gawtam

Sara

How Roboto does it,

